

Module 08 — Seeds and Variables

Tier: ● Working Effectively · Duration: 45 min · Prerequisites: Module 07

Why this module exists: Hardcoded lookup values in SQL — `CASE WHEN country_code = 'DE' THEN 'Germany'` repeated across five models — are a maintenance trap. Change one label, hunt down five files. Seeds eliminate that by turning any lookup table into a version-controlled CSV that every model can `ref()`. Variables eliminate hardcoded constants like `WHERE created_at >= '2024-01-01'` by making them configurable at runtime.

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
00:00	5 min	Recap Module 07	Confirm docs and sources mental model	Q&A	Answer from memory	—	Ask all 4 prep questions cold. Probe: "where does <code>sources.yml</code> live and what does the <code>freshness</code> block do?" — <code>freshness</code> connects to today's variables content (date thresholds).	All 4 correct
00:05	10 min	Seeds: theory	Know what a seed is, where it lives, and when to use it	Present	Annotate decision table	This doc	Walk through the <code>seeds/</code> folder structure. Contrast with source and dim table. The key insight: a seed is for data you control and update via Git, not via pipelines.	"Name two cases where you'd use a seed instead of a source"
00:15	10 min	Variables: theory	Know <code>{{ var() }}</code> syntax, defaults, and runtime override	Present	Write syntax from memory	This doc	Cover <code>var()</code> with default, <code>dbt_project.yml</code> defaults, and <code>--vars</code> runtime flag. Emphasise: variables solve the same problem	"What does <code>{{ var('start_date', '2024-01-01') }}</code> output when no <code>--vars</code> flag is passed?"

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
							seeds solve for lookup data — they remove hardcoded constants from your SQL.	
00:25	10 min	Live demo	See seeds and vars working end-to-end in the exercise project	Demo	Watch and note commands	VS Code + terminal	Run <code>dbt seed</code> , show the DuckDB table created, then show a <code>{{ var() }}</code> substitution in compiled SQL using <code>dbt compile</code> .	"Where does the compiled output appear?"
00:35	5 min	Exercise	Apply both concepts to real models	Practice	Solo exercise	Exercise below	The seed task (Task 1–2) is quick. Task 3 (variables) is the main exercise — circulate actively. Most common confusion: changing the WHERE clause but forgetting to add the var to <code>dbt_project.yml</code> , so <code>dbt run</code> fails with "variable not defined".	All 3 tasks complete, bonus optional
00:40	5 min	Debrief + prep questions	Consolidate and preview Module 09	Debrief	Verbal	Whiteboard	Ask: "What is the difference between a seed and a dbt variable?" — they solve different problems. Seed = lookup data. Variable = runtime constant.	—

Content

Part A — Seeds

Seeds: Two Patterns

Not all seeds serve the same purpose. This module focuses on **lookup seeds**, but the exercise project also uses a second kind — development seeds — which are scaffolding for local work only.

Lookup seeds (production use)

Static reference tables like `country_codes.csv` or `product_categories.csv`. A human curates them, they change rarely, and they are version-controlled in Git. Every medallion layer (Bronze, Silver, Gold) can join to them via `{{ ref() }}`. These are the real seeds you will build and maintain in production.

Development seeds (local DuckDB only)

Files like `raw_contacts.csv`, `raw_deals.csv`, and similar files in the exercise project. These simulate the Bronze layer so that exercises can run locally without a Snowflake connection. In real production, those tables would be loaded by a Lambda pipeline — they are not lookup seeds and the pattern does not carry over to Snowflake.

This module teaches lookup seeds. The development seeds in the exercise project are just scaffolding so the exercises compile.

What a seed is

A seed is a CSV file in the `seeds/` folder of your dbt project. Running `dbt seed` loads it into your data warehouse as a table. Every model can reference it with `{{ ref('seed_name') }}` exactly like any other model.

Seeds are version-controlled. The CSV lives in Git. That means changes to lookup data go through code review, show up in `git diff`, and are rolled back with `git revert` — just like model logic.

Where seeds live

```
dbt_project/
├── seeds/
│   ├── country_codes.csv      ← lookup seed
│   └── product_categories.csv ← lookup seed
├── models/
│   └── ...
```

A seed CSV

```
country_code,country_name,region
DE,Germany,DACH
AT,Austria,DACH
CH,Switzerland,DACH
FR,France,Western Europe
GB,United Kingdom,Northern Europe
US,United States,North America
```

Running seeds

```
# Load all seeds
dbt seed
```

```
# Load one specific seed
dbt seed --select country_codes

# Force a full reload (drops and recreates the table)
dbt seed --full-refresh
```

After `dbt seed`, your warehouse contains a table named `country_codes` in your dev schema. You reference it in models as:

```
SELECT *
FROM {{ ref('country_codes') }}
```

Configuring seeds in `dbt_project.yml`

Seeds inherit the same config system as models. You can set schema, tags, column type overrides, and grants:

```
seeds:
  your_project:
    +schema: seeds          # loads to <target_schema>_seeds instead of default schema
    +tags: ['reference']
    country_codes:
      +column_types:
        country_code: varchar(2)
        country_name: varchar(100)
        region: varchar(100)
```

Column type overrides matter because CSV files have no type information. dbt infers types from the first few rows — a string like `01` becomes integer `1` without an explicit type override. Leading zeros in country codes, postal codes, or product IDs are silently dropped, causing join failures downstream. Always define explicit types for any code column.

When to use a seed vs. a source vs. a dim table

Situation	Use...	Because...
Static lookup data you maintain in code (country codes, status labels, category mappings)	seed	It's version-controlled, small, and changes only when you decide to change it.
Raw data loaded by an external pipeline (HubSpot contacts, deals, prescriptions)	source	You don't control this data — you only observe it. The pipeline owns it.
A dimension that derives from source data and requires transformation logic (SCD, hashing, cleaning)	dim table	It's built by dbt from raw source data, not manually curated.
Large lookup data that is itself produced by a pipeline (e.g., a product catalogue from an ERP API)	source → staging → dim	Even if it's "reference data", if a pipeline populates it, it's a source.

The key question: did a human curate this in a spreadsheet, or did a pipeline produce it?

- Human curated → seed
- Pipeline produced → source

Seeds and the medallion architecture: Seeds are not part of the Bronze / Silver / Gold layers. They are reference data that any layer can join to. In `dbt_project.yml`, configure them into their own schema with `+schema: seeds` so they land in `<target_schema>_seeds` and are clearly separated from medallion tables. A Gold mart joining `country_codes` is still Gold — the seed is just a dimension it references.

Seeds in the exercise project

The exercise project uses two lookup seeds:

```
seeds/
├─ country_codes.csv      ← ISO country code → country name + region
└─ product_categories.csv ← category_id → category label + tier
```

The raw data seeds (`raw_contacts.csv`, `raw_deals.csv`, etc.) are Bronze-equivalent data seeds used for local development. Those simulate the Bronze layer — they are not lookup seeds and not the same concept.

Part B — Variables

What variables solve

Without variables, you repeat constants in SQL:

```
-- fct_deal.sql
WHERE close_date >= '2024-01-01'

-- fct_prescription.sql
WHERE prescribed_at >= '2024-01-01'

-- mrt_deals_funnel.sql
WHERE deal_created_at >= '2024-01-01'
```

Change the start date → edit three files. Miss one → inconsistent results. A variable replaces the constant in every model with a single definition.

`{{ var() }}` syntax

```
-- Without a default – errors if not set
WHERE close_date >= '{{ var("start_date") }}'

-- With a default – safe to run without --vars
WHERE close_date >= '{{ var("start_date", "2024-01-01") }}'
```

The second argument is the fallback value. Always provide a default for variables used in models that run in production — otherwise a missing `--vars` flag breaks your pipeline.

Setting defaults in `dbt_project.yml`

```
vars:
  start_date: '2024-01-01'
  min_deal_amount: 0
  env: 'dev'
```

Variables defined in `dbt_project.yml` are available to all models. The runtime `--vars` flag overrides them:

```
# Use the dbt_project.yml default
dbt run --select fct_deal

# Override start_date at runtime
dbt run --select fct_deal --vars '{"start_date": "2023-01-01"}'

# Override multiple variables
dbt run --vars '{"start_date": "2023-01-01", "min_deal_amount": 500}'
```

Variables in practice: incremental date filters

The most common use of variables in this project is controlling the lookback window for ad-hoc rebuilds:

```
-- fct_deal.sql
{{ config(
  materialized = 'incremental',
  unique_key   = 'deal_key',
  on_schema_change = 'sync_all_columns'
) }}

SELECT
  deal_key,
  deal_id,
  deal_name,
  amount,
  close_date
FROM {{ ref('stg_hubspot__deals') }}

{% if is_incremental() %}
WHERE close_date >= '{{ var("start_date", "2024-01-01") }}'
{% endif %}
```

On a normal incremental run you pass no `--vars`. The default `'2024-01-01'` applies. For a historical reload you override:

```
dbt run --select fct_deal --full-refresh --vars '{"start_date": "2022-01-01"}'
```

Variables for environment-specific config

```
# dbt_project.yml
vars:
  env: 'dev'
```

```
-- Use env to conditionally limit rows in dev
{% if var('env', 'dev') == 'dev' %}
  LIMIT 1000
{% endif %}
```

This pattern speeds up dev runs without touching the model logic for production.

Live Demo Script

Estimated time: 10 minutes.

Step 1 — Show the seeds folder

Open the exercise project in VS Code. Show `seeds/country_codes.csv` and `seeds/product_categories.csv`. Point out: these are plain CSV files, tracked in Git.

Step 2 — Run `dbt seed`

```
dbt seed
```

Show the terminal output. Each seed should show `OK created seed`. Point out the schema it loaded into (`main` for DuckDB, or the target schema for Snowflake).

Step 3 — Query a seed in DuckDB

```
dbt run --select mrt_country_summary
```

Or open a DuckDB connection and run:

```
SELECT * FROM country_codes LIMIT 5;
```

Show that the table exists and has the expected rows.

Step 4 — Show a variable substitution

Open `fct_deal.sql` . If the `{{ var('min_deal_amount', 0) }}` filter is already in place (after Task 3), run:

```
dbt compile --select fct_deal
```

Open `target/compiled/<project>/models/silver/fct_deal.sql` . Show that `{{ var('min_deal_amount', 0) }}` has been replaced with `0` .

Then run with an override and recompile:

```
dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'
```

Show that `target/compiled/` now shows `500` . This makes the substitution visible and concrete.

Exercise (5 min)

Project context: The exercise project has `seeds/country_codes.csv` and the full staging and Silver layer from Modules 01–07. You'll load the seeds, write a new Gold mart, and add a variable to an existing Silver model.

Task 1 — Run `dbt seed` and inspect the output

Run `dbt seed` and confirm that both seeds loaded successfully. Then run:

```
dbt show --select country_codes --limit 10
```

Verify the output contains country codes, country names, and regions. Answer in one sentence: why is `country_codes` a seed and not a source?

Task 2 — Write `mrt_country_summary.sql`

Create `models/gold/mrt_country_summary.sql` .

This Gold mart should:

- Join `dim_contact` with the `country_codes` seed on `country_code`
- Group by `country_name` and `region` (from the seed)
- Count distinct contacts per country as `contact_count`
- Use `materialized='table'` (Gold convention)

Expected columns: `country_code` , `country_name` , `region` , `contact_count` .

Start simple: get the join working first, then add aggregations.

► Expected model (one approach)

Run `dbt run --select mrt_country_summary` to verify it builds without errors.

Task 3 — Add a `min_deal_amount` variable to `fct_deal.sql`

1. Open `dbt_project.yml` and add a `vars` block with a default. The `vars:` key goes at the **root level** of the file — not nested inside `models:` or `seeds:`.

Before:

```
name: 'dbt_training'
version: '1.0.0'

models:
  dbt_training:
    +materialized: view
```

After:

```
name: 'dbt_training'
version: '1.0.0'

vars:
  min_deal_amount: 0

models:
  dbt_training:
    +materialized: view
```

1. Open `models/silver/fct_deal.sql`. Find the WHERE clause (or add one if missing) and add a filter:

```
WHERE amount >= {{ var('min_deal_amount', 0) }}
```

1. Run the model with the default to confirm it works:

```
dbt run --select fct_deal
```

1. Override the variable at runtime to filter to deals above €500:

```
dbt run --select fct_deal --vars '{"min_deal_amount": 500}'
```

1. Run `dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'` and open `target/compiled/` to confirm `500` appears in the compiled SQL.

Bonus — Find a place where a seed would improve existing code

Look through the existing staging or Silver models. Find one place where a lookup value is hardcoded in a CASE WHEN or WHERE clause that would be better managed as a seed CSV. Write down:

- Which model contains it
 - What the seed file would look like (column names + 3 sample rows)
 - How you'd rewrite the model to use `{{ ref('your_seed') }}`
-

Reference Material

- [dbt seeds docs](#)
 - [dbt variables docs](#)
 - [dbt var\(\) function reference](#)
 - [dbt seed command reference](#)
 - Internal: `bloomwell-conventions` skill — covers when lookup data belongs in seeds vs. dim tables
-

Prep Questions for Module 09

1. What is the difference between `{{ }}` and `{% %}` in Jinja? Give one example of each in a dbt model.
2. You want to write a block of SQL that only runs during an incremental model's subsequent runs. What Jinja construct do you use?
3. What does `{{ ref('stg_hubspot__deals') }}` compile to in your dev environment?
4. If you needed to loop over a list of columns and generate SQL for each one, which Jinja tag would you use — `{{ }}` or `{% %}` ?